

CI/CD & Jenkins Integration

What is CI/CD? (Simple Meaning)

CI/CD is a way of **automating the entire software delivery process** so that whenever developers make changes to code, the system automatically:

- ✓ Builds the application
- ✓ Tests it
- ✓ Integrates it with existing code
- ✓ Packages it
- ✓ Deploys it
- ✓ Gives feedback

This makes software delivery **fast, reliable, and repeatable** — without depending on manual work.

1. Delivery Pipeline Concepts

1.1 What is a Delivery Pipeline?

A **Delivery Pipeline** is a series of *automated steps* that code goes through from a developer's laptop to the final running software.

Think of it like a factory production line for software.

Typical Pipeline Stages

1. **Source Code Commit** — Developers push code into Git
2. **Build** — Code is compiled
3. **Unit & Integration Testing** — Automated program logic tests run
4. **Automation Testing** — UI & functional tests run
5. **Package / Artifact Creation** — Software bundle is created
6. **Deployment** — Application is deployed to server
7. **Monitoring & Feedback** — Check performance & errors

Main Goal of CI/CD

Deliver software:

- ✓ Faster
 - ✓ With fewer errors
 - ✓ In a repeatable way
 - ✓ With minimum human effort
-

1.2 Continuous Integration (CI)

Continuous Integration means:

Developers **frequently upload (commit) code to a shared repository like Git.**

Every time code is uploaded:

- ✓ The code is automatically built
- ✓ Tests run automatically
- ✓ Issues are detected immediately

Benefits of CI

- Defects are found early
- Developers get fast feedback
- Code always stays in a working condition
- Reduces conflicts when merging code

Simple Memory Line:

CI = *"Code changes are continuously integrated and tested."*

1.3 Continuous Delivery (CD)

Continuous Delivery means:

Software is always kept in a **ready-to-deploy state.**

Deployment to production **may still need manual approval.**

Example:

Software is packaged and tested → The release manager clicks "Deploy".

1.4 Continuous Deployment

This is the **fully automatic version of Continuous Delivery.**

Every build that passes tests is deployed to production **without human approval.**

Used in:

- Large web apps
- SaaS tools
- Cloud platforms

Example: Netflix → Updates many times daily.

1.5 Why CI/CD Matters

CI/CD helps organizations achieve:

- ✓ Faster software releases
- ✓ More automation
- ✓ Fewer bugs

- ✓ Early feedback
- ✓ Less manual work
- ✓ Better teamwork
- ✓ Higher product quality

That's why **CI/CD is a core part of DevOps.**

2. Jenkins — Overview

2.1 What is Jenkins?

Jenkins is the world's most popular CI/CD automation server.

It is used to:

- Build software
- Run automated tests
- Deploy applications
- Orchestrate DevOps workflows

And it works with almost every DevOps tool.

2.2 Key Features of Jenkins

- Huge plugin ecosystem
 - Supports CI/CD pipelines
 - Integrates with Git, Maven, Docker, Kubernetes, Selenium, etc.
 - Can run builds on multiple machines
 - Can schedule builds
 - Provides dashboards and reports
 - Managed fully via web browser
- 
-

3. Jenkins Management — High-Level Understanding

A typical Jenkins setup includes:

- **Jenkins Server (Master Node)** — Controls everything
- **Plugins** — Add features
- **Tool Integrations** — Git, JDK, Maven etc.
- **User Roles & Permissions** — Security control

The **Dashboard** is used for:

- ✓ Creating jobs
 - ✓ Running pipelines
 - ✓ Monitoring builds
-

4. Slave Node (Agent) Setup

To avoid overloading one machine, Jenkins allows execution on **multiple machines called Agents (or Slave Nodes)**.

Why use Nodes?

- Share workload
- Speed up processes
- Run builds on different OS platforms
- Improve performance

Node Setup Conceptual Steps

1. Prepare the machine
2. Install Java / runtime
3. Connect the machine to Jenkins
4. Assign labels
5. Bind jobs to nodes

Agents can be:

- Windows systems
- Linux servers
- Cloud machines
- Docker containers



5. Creating Pipelines in Jenkins

A **Pipeline** = The automated workflow defined in Jenkins.

There are 2 main types:

5.1 Freestyle Jobs

- Built using GUI
- Best for simple tasks
- Easy for beginners

5.2 Scripted / Declarative Pipelines

Written as **code** using a file called Jenkinsfile

Pipeline stages may include:

- Pull code from Git
- Build application
- Run tests
- Package
- Deploy
- Send notifications

Benefits of Pipeline-as-Code

- ✓ Stored in version control
 - ✓ Easier to reuse
 - ✓ Easy to review
 - ✓ Portable
-

6. Selenium — Jenkins Integration

This means **Selenium test scripts run automatically inside the CI/CD pipeline.**

6.1 Why integrate Selenium with CI/CD?

Because it allows:

- ✓ Continuous regression testing
 - ✓ Build stability verification
 - ✓ Reduced manual work
 - ✓ Continuous software validation
-

6.2 How Integration Works

1. Developer uploads code
2. Jenkins automatically starts the pipeline
3. Application is built & deployed
4. Selenium tests run automatically
5. Jenkins collects & displays results
6. Build is marked **Passed / Failed**

If tests fail → alerts & investigation follow.

6.3 Typical Selenium CI Flow

Pipeline normally includes:

1. Checkout test code
2. Install dependencies
3. Run tests
4. Generate reports
5. Publish test results
6. Notify team

This creates a **continuous feedback loop**.

7. CI/CD Best Practices

To make pipelines effective:

- Commit code frequently
- Keep main branch stable
- Automate everything
- Run quick smoke tests first
- Run deep regression later
- Stop pipeline immediately on failure
- Maintain logs & traceability

Security Best Practices

- Role-based access
 - Secret & credential encryption
 - Secure artifact storage
-

8. CI/CD & DevOps Alignment

DevOps Goal	How CI/CD Helps
Faster delivery	Automated pipelines
Better collaboration	Shared workflow
Higher quality	Continuous testing
System reliability	Repeatable builds
Fast feedback	Build/test reports
Reduced risk	Small frequent releases

CI/CD is not “just a tool” — it is a business capability.

9. Summary — Why CI/CD + Jenkins Matter

CI/CD enables:

- ✓ Continuous code integration
- ✓ Automated testing with Selenium
- ✓ Reliable & repeatable builds
- ✓ Confident releases
- ✓ Distributed execution
- ✓ Governance via pipeline-as-code

Jenkins works as the central brain — orchestrating the entire DevOps workflow.

Together:

→ CI/CD

+

→ Jenkins

+

→ Selenium

create a **complete, quality-driven delivery system.**

